SkillSwap: Peer-to-Peer Learning Platform Documentation

1. Project Overview

SkillSwap is a premium peer-to-peer learning ecosystem designed to facilitate bidirectional skill sharing. It uses a sophisticated matching algorithm and verified testing to ensure high-quality learning experiences.

---

2. Key Features

🚀 User Experience & Profiles

- **Dynamic Profiles**: Users can list skills they want to teach and skills they want to learn.
- **Availability Management**: Interactive scheduling for session planning.
- **Resume Integration**: Upload and parse resumes to extract certifications and verify expertise.

🛡️ Verification & Testing (Test Portal)

- **Skill-Based Tests**: Users must pass a 15-question rigorous test (score >= 10/15) before being allowed to teach a skill.
- **Verification Levels**: Proficiency is categorized (Beginner to Expert) based on test performance and self-reports.
- **Static & AI Banks**: Support for both manual question banks and dynamic AI generation.

🤝 Advanced Matching Engine

- **Bidirectional Matching (Perfect Swap)**: Finds users who want exactly what you teach and vice-versa.
- **Multi-Factor Scoring**:
  - Test Scores (30% weight)
  - Skill Level Alignment (20% weight)
  - Availability Overlap (10% weight)
  - Reputation & Certifications (20% weight)
- **Teacher Recommendations**: Instant discovery of top-rated experts for specific skills.

💬 Collaboration & Sessions

- **Real-Time Presence**: Track when users are online for sessions.
- **Session Agendas**: Structured learning with pre-defined goals.
- **Integrated Messaging**: Secure chat for coordinating swaps.
- **Feedback & Reputation**: Post-session ratings that influence global matching scores.

---

3. Core Algorithms: Deep Dive

🧠 1. Weighted Skill Matching Algorithm

The heart of SkillSwap is a multi-dimensional matching engine that calculates a **Match Compatibility Score (MCS)** between 0.0 and 1.0.

Component	Weight	Logic

Mandatory Skill Test	**30%**	Requires a verified score (>= 67%) in the specific skill via the Test Portal.
Proficiency Alignment	**20%**	Uses a "Bidirectional Gap" logic. Peer matching (same level) or Mentor matching (1 level above) yields a 1.0 score.
Goal Alignment	**10%**	Maps learning goals (e.g., Job Prep) to teaching motivations (e.g., Love Teaching) using a heuristic compatibility matrix.
Availability Overlap	**10%**	Analyzes 3-hour time slots across the week to find the max intersection of free time.
Reputation Score	**10%**	A rolling average of ratings from previous sessions (Normalized 1-5 scale to 0-1).
Certifications	**10%**	Credibility scores extracted from parsed professional certifications.
Soft Factors	**10%**	Combines Time Commitment (5%) and Learning Style (5%) compatibility (e.g., Pair Programming vs. Video Call).

🛠️ 2. The "Bidirectional Gap" Logic

Unlike simple keyword matching, our algorithm explicitly values "Peer Learning":

- **Gap = 0 (Peer)**: Score 1.0 (Ideal for collaborative practice).
- **Gap = +1 (Mentor)**: Score 1.0 (Ideal for structured learning).
- **Gap = +2 (Expert)**: Score 0.9.
- **Gap < 0 (Reverse)**: Score 0.4 (Learner is more advanced than the teacher).

---

4. Artificial Intelligence Implementation

🤖 Architecture: Interface-Driven AI

SkillSwap uses an abstraction layer to remain **Model-Agnostic**, allowing seamless switching between **Google Gemini** and **OpenAI**.

A. AI Question Generation (`AIQuestionGeneratorService`)

- **Process**: When a user starts a test, the system prompts the AI with the skill name and requested difficulty (HARD).
- **Output**: The AI generates 15 high-order thinking questions, distractors, and a JSON-formatted answer key.
- **Fallback**: To ensure zero-downtime, the system uses a **Static Question Bank** (e.g., for Java, Spring Boot) when AI limits are reached.

B. Sentiment & Quality Analysis (`SentimentAnalyzer`)

- Uses Natural Language Processing (NLP) to analyze post-session feedback.
- Detects subtle trends in session quality that simple star ratings might miss.
- Influences the **Reputation Score** by weighing feedback based on sentiment intensity.

C. Behavioral Recommendation Console (`RecommendationEngine`)

- Implements a "Learn" loop where the system tracks user interactions (CLICKS, SESSION_COMPLETIONS).

- Refines future teacher suggestions based on what type of mentors the user successfully learns from.

---

5. Recent Technical Changes

- **Dashboard Overhaul**: Split view for "Received Requests" (Teacher) and "Sent Requests" (Learner) for better workflow.
- **Notification System Fixes**: Standardized UI/Backend communication by fixing field naming (isRead vs read).
- **Session Acceptance UI**: New interface for accepting or rejecting session requests with immediate feedback.
- **API Optimization**: Implemented `JOIN FETCH` in JPA queries to resolve Hibernate lazy loading issues in session management.
- **JWT Fixes**: Enhanced token validation and OTP (send/resend) flow for smoother onboarding.

---

5. Technical Stack

- **Backend**: Spring Boot 3.x, MySQL, JUnit 5.
- **Frontend**: Vite + React, TypeScript, Tailwind CSS, Lucide Icons.
- **Design System**: Shadcn/UI for a clean, professional aesthetic.